

# Rust

WPC

2026 年 1 月 4 日

## 目录

|                  |           |
|------------------|-----------|
| <b>1 Rust 类型</b> | <b>2</b>  |
| 1.1 Rust 安装      | 2         |
| 1.2 数值类型         | 2         |
| 1.3 文本类型         | 3         |
| 1.4 引用           | 3         |
| 1.5 元组、数组和向量     | 3         |
| 1.6 结构体          | 3         |
| 1.7 枚举与模式        | 4         |
| 1.8 类型操作         | 6         |
| <b>2 Rust 语法</b> | <b>7</b>  |
| 2.1 表达式          | 7         |
| 2.2 所有权          | 7         |
| 2.3 错误处理         | 7         |
| 2.4 模块           | 7         |
| <b>3 特型与泛型</b>   | <b>9</b>  |
| 3.1 泛型           | 9         |
| 3.2 运算符重载        | 9         |
| 3.3 实用工具特型       | 10        |
| <b>4 集合与迭代器</b>  | <b>12</b> |
| 4.1 闭包           | 12        |
| 4.2 集合           | 12        |
| 4.3 迭代器          | 12        |
| <b>5 并发</b>      | <b>14</b> |
| 5.1 并发           | 14        |

## 1 Rust 类型

### 1.1 Rust 安装

通过 `rustup` 安装 Rust 后，会自动配置 Rust 的包管理器 [Cargo](#)、编译器 `rustc` 以及文档生成工具 `rustdoc`。 [std 文档](#)。注意：如果是 Windows，需要额外下载 [c++编译工具](#)。

安装 Rust 写的二进制工具可以使用 `cargo-binstall`，自己写 cli 工具，打包可以用 `cargo-dist` 工具。

dist 使用示例

```
dist init
git add .
git commit -am 'chore: wow shiny new dist CI!'
git push

# 真正触发 GitHub CI / Release 的是 tag
git tag v0.1.0
git push --tags

# 可选，本地构建
dist build
dist plan
```

### 1.2 数值类型

Rust 的其他数值类型与大多数语言类似，不过值得注意的是：Rust 将 `u8` 类型视为字节类型，字节字面量用 `b'x'` 表示，它对应一个 `u8` 值；可以使用 `as` 将 `u8` 转换为 `char` 类型。如果需要扩展的数值类型，可以使用 `num` 库。Rust 提供了一些常用的数学常量在 [std::f32::consts](#) 和 [std::f64::consts](#) 中。

当整型算术溢出时，`debug build` 会 `panic`，而 `release build` 会回绕，如果这不是你期望的行为，Rust 提供了多种显式的整型运算方法 (`add`、`sub`、`mul`、`div`、`rem`、`neg`、`abs`、`pow`、`shl`、`shr`)：

- 检查运算 (`checked`)：返回结果的 `Option` 值： `Some(v)` 或者 `None`。
- 回绕运算 (`wrapping`)：如果结果溢出，返回取模后
- 饱和运算 (`saturating`)：返回可表达的最接近的值，例如 `128_u8.saturating_mul(3)==255`。
- 溢出运算 (`overflowing`)：返回元组，第一个元素是回绕后的结果，第二个是 `bool` 值表示是否溢出。

## 1.3 文本类型

Rust 字符类型 `char` 以 32 位表示 unicode 字符，仅允许 `u8` 类型通过 `as` 转换为 `char`。Rust 的字符串类型 `String` 则是一个 UTF-8 编码的字节序列，因此 `len` 返回字节数而不是字符数。类型关系上，`&str` 类似 `&[T]`，`String` 则类似 `Vec[T]`。Rust 还支持原始字符串字面量：`r""`。创建 `String` 的常见方式：`to_string`、`to_owned`、`format!`、`concat` 和 `join`。

## 1.4 引用

Rust 中的引用类似 C/C++ 中的指针，不过引用总是有效的，而且对引用赋值会让引用指向另一个值，而不是改变引用指向的值本身。对引用进行 `.` 运算符操作会隐式借用或解引用。Rust 允许对任意表达式进行引用操作。

Rust 会为每个引用类型分配一个生命周期，这是在编译期间虚构的产物，没有运行时表示。生命周期存在两个约束：

- 值的生命周期必须比引用的生命周期长。
- 引用必须在被初始化到最后一次使用的生命周期内都能访问。

函数签名的时候可以使用生命周期标注来指定引用的生命周期，其语法类似泛型的语法。例如：`fn g<'a>(p: &i32)`，这个函数的参数 `p` 的生命周期被标注为 `'a`，意思是能接受任意涵盖函数生命周期的引用。如果引用类型出现在另一个类型的定义中，那么必须写出它的使用寿命。

## 1.5 元组、数组和向量

Rust 元组中元素数据类型可以不同，不过只能使用常量索引元组，`()` 表示零元组，通常用作空返回值。类型 `[T;N]` 表示元素类型为 `T`、长度为 `N` 的数组，数组索引必须为 `usize`。类型 `Vec<T>` 表示向量。`&[T]`、`&mut [T]` 分别表示共享切片与可变切片，切片包含两个值：指针和长度。

`vec![2, 3, 4]` 创建向量，`[0u8,1024]` 创建一个 1KB 的数组缓存区。数组和向量的大部分方法一般是在切片上定义的，对数组和向量使用方法时，会隐式生产一个切片。

## 1.6 结构体

Rust 有 3 种结构体类型：具名字段型结构体、元组型结构体和单元型结构体。

## 结构体定义和构造

```
struct GrayscaleMap {
    pixels: Vec<u8>,
    size: (usize, usize)
}

let image = GrayscaleMap {
    pixels: vec![0; 1024 * 576],
    size: (1024, 576)
};

struct Ascii(Vec<v8>); //元组结构体主要用来创造新类型
struct Onesuch; //单元型结构体，类似零元组
```

用 `impl` 定义关联函数时，第一个参数是结构体本身，可以传递 `self`、`&self`、`&mut self`，甚至是 `Box<Self>`、`RC<Self>` 或者 `Arc<Self>`。通过 `.` 运算符使用方法时，Rust 会自动获取对应类型的引用。也可以在 `impl` 块中定义类型关联函数（类似静态方法）、关联常量。

当需要一个不可变结构体里面的某个数据可变时，数据可使用 `std::cell` 模块中的 `Cell<T>` 或者 `RefCell<T>` 定义，`Cell` 使用 `get` 和 `set` 方法获取和修改值，`RefCell` 借用引用，使用 `try_borrow`、`borrow` 和 `borrow_mut` 等方法。

## 1.7 枚举与模式

枚举在开发中被广泛用于表示结构化数据，例如 [JSON](#)。与其他语言的枚举不同，Rust 的枚举有元组变体和结构体变体，可以携带数据。如果你需要从整数到枚举的转换，可以使用 [enum\\_primitive](#) 库。枚举还通常与 `match` 一起使用，进行模式匹配。

## 枚举的定义

```
enum Shape {
    Circle(f64),
    Rectangle { width: f64, height: f64 },
}

enum Option<T> {
    None,
    Some(T),
}

enum Result<T,E> {
    OK(T),
    Err(E),
}
```

| 模式类型    | 示例                       | 说明                 |
|---------|--------------------------|--------------------|
| 字面量     | 42, "hello"              | 匹配与字面量相等的值，也可匹配常量。 |
| 通配符     | _                        | 忽略某个值。             |
| 占位符     | _x                       | 绑定但不使用该变量，避免编译器警告。 |
| 变量      | name, mut count          | 将值绑定到变量上。          |
| 引用      | &x, & mut y              | 匹配引用类型的值，同时解引用和绑定。 |
| 引用变量    | ref field, ref mut field | 借用对匹配值的引用          |
| 子模式绑定   | val @ 0 ..=99            | 使用@左边的变量名，匹配右边的模式  |
| 元组模式    | (x, y), (_, b)           | 可忽略部分元素。           |
| 结构体     | Account {id, name, ..}   | 可按字段名解构。           |
| 枚举模式    | Some(value), None        | 匹配枚举变体并解构其携带的数据。   |
| 数组/切片模式 | [first, _, last]         | 匹配固定长度数组或切片前后元素。   |
| 范围模式    | 1 ..=5, 256.., 'a'..'k'  | 匹配落在指定范围内的值。       |
| 匹配守卫    | x if x > 5               | 在匹配时添加条件判断。        |
| 多重模式    | 1   2   3                | 匹配多个可能的模式之一。       |
| 嵌套模式    | Some((x, y))             | 支持在结构内部继续进行模式匹配。   |
| 剩余模式    | [head, tail @ ...]       | 匹配剩余部分的数据（如切片或数组）。 |

表 1.1: Rust 模式匹配类型汇总表

## 1.8 类型操作

`!`类型是一个特殊类型，通常用于表示函数不会正常返回。用 `type` 定义类型别名，若要从堆中分配值使用 `Box::new`。在 Rust 中，有几类重要的自动类型转换，被称为隐式解引用，适用于所有实现 `Deref` 特型的类型：

- `&String` 会自动转换为 `&str`。
- `&Vec<i32>` 会自动转换为 `&[i32]`。
- `&Box<Chess>` 会自动转换为 `&Chess`。

## 2 Rust 语法

### 2.1 表达式

Rust 中没有语句，只有表达式。块表达式的值是块中最后一个表达式的值，if 表达式的所有块都必须生成同样类型的值，if let 表达式是 match 表达式只有两种情况的简写，通常使用 if let 从 Option 或者 Result 中获取数据，也有 while let 表达式。loop 表达式用来编写无限循环。

### 2.2 所有权

在 Rust 中，为变量赋值、函数参数传递和函数返回时，所有权会被移动。

- 大多数类型的赋值会将值转移给目标，而源会回到未初始化状态。
- 变量离开作用域时，值会被丢弃（drop）。
- 通过引用传递变量时，所有权不会被移动。
- 可以使用 clone() 方法来显式复制数据。

注意并非所有类型值的拥有者都能变成未初始化状态，例如数组和向量类型，不能将其中某个索引处的值移动走。如果你真的要实现这样的效果，建议创建一个包含 Option<T>字段的结构体，然后将该字段设置为 None，Rust 专门为这种操作定义了一个函数，std::mem::take，它会将字段值替换为默认值，并返回原来的值。

有些类型实现了 Copy trait，这些类型的值在赋值时不会移动所有权，而是进行复制。

- 所有整数类型、浮点数类型、char 类型和 bool 类型都是 Copy 类型。
- 所有共享引用类型都是 Copy 类型。
- 元组类型当且仅当其所有元素类型都是 Copy 类型时，才是 Copy 类型。

Rust 通过引用计数指针类型 Rc<T>和 Arc<T>来实现共享所有权，他们之间的唯一区别就是 Arc 可以安全地在多线程间共享。

### 2.3 错误处理

Rust 的错误处理使用 panic 和 Result。当数组越界、整数除以 0、断言失败或者在 Err 的 Result 上调用 expect 或 unwrap 时，程序会调用 panic 宏并中止执行。panic 宏会打印错误信息和调用栈，然后展开栈并清理资源，最后终止线程，如果线程为主线程，程序也会终止。

建议使用 expect 代替 unwrap，如果有默认值的话，可以使用unwrap\_or 和unwrap\_or\_else。如果需要传播错误，可以使用?运算符，不过它只能用于返回 Result 或 Option 的函数中，如果需要自定义错误类型，可以考虑使用 thiserror 库，如果需要更强大的错误处理，可以使用 anyhow 库。

### 2.4 模块

使用 cargo 编译库代码时，使用--crate-type lib 选项，会告诉编译器不去寻找 main 函数，而是生成一个 rlib 文件。cargo build --release 会生成优化过的程序，运行更快，编译时间更

长。Cargo.toml 文件中可以指定 crate 的名称、版本、作者等信息，不同编译命令对应的区段不同。

| 编译命令                  | 区段                |
|-----------------------|-------------------|
| cargo build           | [profile.dev]     |
| cargo build --release | [profile.release] |
| cargo test            | [profile.test]    |

表 2.1: Cargo 对应 Profile 区段

pub 声明可以导出语法项，pub(crate) 表示在当前 crate 内可见，pub(super) 表示在父模块可见，任何未标记为 pub 的内容都只能在定义它的模块及其子模块内使用。

子模块声明语法为 mod spores;，这会让编译器检查 spores.rs 和 spores/mod.rs 文件。如果把子模块按文件夹组织，则可以在文件夹中添加 mod.rs 文件来声明子模块：pub mod spores;。

crate 指的是当前模块所在的 crate，super 指的是父模块，self 指的是当前模块。可以通过 use super::\* 访问父模块中的私有语法项。模块和文件系统有些类似之处：use 创建别名，self 和 super 类似 ./ 和 ../，crate 类似当前项目根目录。如果在模块中使用 pub use 公开了语法项，则可以通过 use prelude::\* 来导入这些语法项。



## 3 特型与泛型

### 3.1 泛型

在 Rust 中，`&mut dyn Write` 表示实现了 `Write` 特型的任意类型值的可变引用。`dyn Write` 被称为特型对象，Rust 会在需要时自动将普通引用转换为特型对象，而且创建特型对象只能通过这种转换。

在实际项目中，如果你需要一个混合类型的集合（例如一个 `Vec` 中存放不同类型的值），通常就需要使用特征对象；其他情况下，使用泛型一般是更好的选择。Rust 中还有一个 `impl Trait` 特性，可以把 `impl Iterator<Item=u8>` 当一个类型来使用。

#### 特型实现

```
impl TraitName for Type {
    // 只能实现 TraitName 中声明的方法
}

impl Type {
    // 辅助函数可以写在这
}

// 子特型定义方法一
trait Creature: Visible {
}

// 子特型定义方法二
trait Creature where Self: Visible {
}

trait Iterator {
    type item; // 关联类型，用来表示 next 的返回值类型
    fn next(&mut self) -> Option<Self::Item>;
}
```

### 3.2 运算符重载

在 Rust 中，表达式 `a + b` 实际上是 `a.add(b)` 的简写形式。其他需要注意的是：

- `!` 对 `bool` 类型表示逻辑取反，对整数类型表示按位取反。
- `+` 的左操作数不能是 `&str`。

- `PartialEq` 中的 `ne` 方法有默认实现，只需要实现 `eq` 方法即可，可以自动派生。
- `x == x` 不一定总为真（比如 NaN），如果需要保证自反性，则应实现 `Eq` 特型，可以自动派生。
- `PartialOrd` 需要实现的方法是 `partial_cmp`，它返回 `Option<Ordering>`，用于部分排序。
- 如果需要全排序，可以实现 `Ord` 特型，它继承自 `Eq`。

| Trait                                                                                                                                                                                                                                                                                                                                                                        | Operator                                                                                                                                                                                                                                             |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>std::ops::Neg</code><br><code>std::ops::Not</code>                                                                                                                                                                                                                                                                                                                     | <code>-x</code><br><code>!x</code>                                                                                                                                                                                                                   |
| <code>std::ops::Add</code><br><code>std::ops::Sub</code><br><code>std::ops::Mul</code><br><code>std::ops::Div</code><br><code>std::ops::Rem</code>                                                                                                                                                                                                                           | <code>x + y</code><br><code>x - y</code><br><code>x * y</code><br><code>x / y</code><br><code>x % y</code>                                                                                                                                           |
| <code>std::ops::BitAnd</code><br><code>std::ops::BitOr</code><br><code>std::ops::BitXor</code><br><code>std::ops::Shl</code><br><code>std::ops::Shr</code>                                                                                                                                                                                                                   | <code>x &amp; y</code><br><code>x   y</code><br><code>x ^ y</code><br><code>x &lt;&lt; y</code><br><code>x &gt;&gt; y</code>                                                                                                                         |
| <code>std::ops::AddAssign</code><br><code>std::ops::SubAssign</code><br><code>std::ops::MulAssign</code><br><code>std::ops::DivAssign</code><br><code>std::ops::RemAssign</code><br><code>std::ops::BitAndAssign</code><br><code>std::ops::BitOrAssign</code><br><code>std::ops::BitXorAssign</code><br><code>std::ops::ShlAssign</code><br><code>std::ops::ShrAssign</code> | <code>x += y</code><br><code>x -= y</code><br><code>x *= y</code><br><code>x /= y</code><br><code>x %= y</code><br><code>x &amp;= y</code><br><code>x  = y</code><br><code>x ^= y</code><br><code>x &lt;&lt;= y</code><br><code>x &gt;&gt;= y</code> |
| <code>std::ops::Index</code><br><code>std::ops::IndexMut</code>                                                                                                                                                                                                                                                                                                              | <code>x[y]</code><br><code>x[y] = z</code>                                                                                                                                                                                                           |
| <code>std::cmp::PartialEq</code><br><code>std::cmp::PartialOrd</code>                                                                                                                                                                                                                                                                                                        | <code>x == y, x != y</code><br><code>x &lt; y, x &gt; y, x &lt;= y, x &gt;= y</code>                                                                                                                                                                 |

表 3.1: Rust 运算符重载特型

### 3.3 实用工具特型

需要注意的是：

| Trait                                                     | Description                             |
|-----------------------------------------------------------|-----------------------------------------|
| <code>std::ops::Drop</code>                               | 提供对象销毁时的自定义行为，类似析构函数。                   |
| <code>std::marker::Sized</code>                           | 标记类型在编译时大小已知，用于泛型约束。                    |
| <code>std::fmt::Debug</code>                              | 用于格式化输出调试信息，通常用于 <code>{:?}</code> 占位符。 |
| <code>std::fmt::Display</code>                            | 用于用户可读的格式化输出，通常用于 <code>{}</code> 占位符。  |
| <code>std::clone::Clone</code>                            | 实现值的显式深拷贝。                              |
| <code>std::marker::Copy</code>                            | 标记类型可按位复制，一般用于小型标量类型。                   |
| <code>std::default::Default</code>                        | 提供默认值，通过 <code>::default()</code> 获取实例。 |
| <code>std::ops::Deref</code> 和 <code>DerefMut</code>      | 提供解引用操作 <code>*</code> ，允许自定义智能指针行为。    |
| <code>std::borrow::Borrow</code> 和 <code>BorrowMut</code> | 用于借用转换，常用于哈希表或集合中。                      |
| <code>std::borrow::AsRef</code> 和 <code>AsMut</code>      | 提供将类型引用转换为另一种引用的通用方法。                   |
| <code>std::convert::From</code> 和 <code>Into</code>       | 提供类型转换方法。                               |
| <code>std::convert::TryFrom</code> 和 <code>TryInto</code> | 提供可失败类型转换方法。                            |
| <code>std::borrow::ToOwned</code>                         | 用于从借用数据创建拥有权类型。                         |

表 3.2: Rust 常用实用工具特型及其描述

- `Drop` 中的 `drop` 方法只能被隐式调用，且 `Drop` 与 `Copy` 不能同时实现。
- `Sized` 是 Rust 中的隐式默认约束，如果希望类型可以不确定大小，需要显式写为 `T: ?Sized`。
- 在泛型代码中，优先使用 `clone_from` 来减少 `clone` 产生的不必要的空间分配和释放。
- 实现了 `Deref` 的类型在使用时会被隐式解引用，但隐式解引用不会自动满足类型约束。
- 如果类型 `T` 实现了 `Default`，那么 `Rc<T>`、`Cow<T>` 等包装类型也会自动实现 `Default`；元组若所有元素都实现了 `Default`，则元组也自动实现；结构体则需要使用 `#[derive(Default)]` 来生成默认实现。
- 给定了 `From` 实现后，标准库会自动为对应类型生成 `Into` 实现。

## 4 集合与迭代器

### 4.1 闭包

闭包与函数类型并不相同，函数类型如 `fn(&City) -> bool`，表示一个具体的函数指针类型；而每个闭包在编译时都有自己独立的匿名类型，不过它们都实现了相应的函数特型，例如 `Fn(&City) -> bool`。

闭包根据捕获方式的不同，会实现不同的特型：消耗捕获变量的闭包实现 `FnOnce`；需要可变访问的闭包实现 `FnMut`；仅通过共享引用访问的闭包实现 `Fn`。`Fn` 是 `FnMut` 的子特型，`FnMut` 是 `FnOnce` 的子特型。

对于非 `move` 闭包，如果它不修改捕获的值，只通过共享引用访问，则闭包类型本身可以实现 `Clone` 和 `Copy`。而对于 `move` 闭包，若其捕获的所有变量都实现了 `Clone` 和 `Copy`，则闭包自身也会自动实现这两个特型。

### 4.2 集合

| 集合                                                 | 描述                                                                                                                   |
|----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <code>Vec&lt;T&gt;</code>                          | 动态数组。                                                                                                                |
| <code>VecDeque&lt;T&gt;</code>                     | 双端队列（可增长的环形缓存区）。                                                                                                     |
| <code>LinkedList&lt;T&gt;</code>                   | 双向链表，其他集合通常是更好的选择。                                                                                                   |
| <code>HashMap&lt;K,V&gt; where K: Eq + Hash</code> | 基于哈希表的键值映射。                                                                                                          |
| <code>BTreeMap&lt;K,V&gt; where K: Ord</code>      | 基于 B 树的有序映射，按键排序。                                                                                                    |
| <code>HashSet&lt;T&gt; where T: Eq + Hash</code>   | 无序集，支持 <code>&amp;</code> 、 <code> </code> 、 <code>-</code> 、 <code>^</code> 、 <code>==</code> 和 <code>!=</code> 操作。 |
| <code>BTreeSet&lt;T&gt; where T: Ord</code>        | 基于 B 树的有序集合，支持有序遍历。                                                                                                  |
| <code>BinaryHeap&lt;T&gt; where T: Ord</code>      | 最大堆。                                                                                                                 |

表 4.1: Rust 集合

### 4.3 迭代器

迭代器是任何实现了 `std::iter::Iterator` 特型的值。而 `IntoIterator` 则表示可被转换为迭代器的类型（即“可迭代者”）。大多数集合类型都提供了 `iter` 和 `iter_mut` 方法，用于生成迭代器。如果某个类型实现了 `IntoIterator` 特型，也可以通过调用 `into_iter` 方法直接创建迭代器，要注意的是：表达式 `favorites.iter()` 等价于 `(&favorites).into_iter()`。

`from_fn` 可以根据提供的函数生成迭代器，如果后续值依赖前一个值，则应使用 `successors`，这两个函数用于生成迭代器，应作为最后的选择。此外，`drain` 方法也能通过“抽取”集合中的元素来生成迭代器。需要注意的一点是，`Iterator` 特型提供了 `size_hint` 方法，用于返回迭代器可能产生元素的数量范围。

| 迭代器适配器                |                                                                    |
|-----------------------|--------------------------------------------------------------------|
| map                   | 对每个元素应用函数并返回新的迭代器。                                                 |
| filter                | 过滤满足条件的元素。                                                         |
| filter_map            | 同时进行过滤与映射，自动跳过返回 <code>None</code> 的项。                             |
| enumerate             | 为每个元素附加索引（从 0 开始）。                                                 |
| take、take_while       | 分别迭代前 N 个元素，或在条件为真时持续迭代。                                           |
| skip、skip_while       | 分别跳过前 N 个元素，或在条件为真时跳过元素。                                           |
| chain                 | 串联两个迭代器，依次产出其所有元素。                                                 |
| zip                   | 将两个迭代器配对为元组。                                                       |
| rev                   | 反向迭代（要求实现 <code>DoubleEndedIterator</code> ）。                      |
| peekable              | 允许在消费前预览下一个元素。                                                     |
| cycle                 | 无限重复迭代器的内容。                                                        |
| inspect               | 在每次迭代时执行副作用（常用于调试或打印）。                                             |
| flat_map              | 将每个元素映射为迭代器并展平成单一序列。                                               |
| cloned、copied         | 对 <code>&amp;T</code> 元素返回克隆或拷贝的值。                                 |
| by_ref                | 通过可变引用继续使用原迭代器（避免其被移动）。                                            |
| fuse                  | 使迭代器一旦返回 <code>None</code> ，之后所有调用都返回 <code>None</code> 。          |
| 消耗迭代器                 |                                                                    |
| collect               | 将迭代器收集为集合（如 <code>Vec</code> 、 <code>HashSet</code> 等）。            |
| for_each、try_for_each | 对每个元素执行给定函数。                                                       |
| fold、rfold            | 从初始值开始进行累积计算， <code>rfold</code> 从后向前累积。                           |
| try_fold、try_rfold    | 类似 <code>fold</code> 和 <code>rfold</code> ，但在遇到错误时可提前短路。           |
| count、sum、product     | 分别返回元素数量、求和或求乘积。                                                   |
| min、max               | 返回最小值或最大值（元素需实现 <code>Ord</code> ）。                                |
| min_by、max_by         | 根据提供的比较函数返回最小值或最大值。                                                |
| min_by_key、max_by_key | 根据提供的键函数返回最小值或最大值。                                                 |
| any、all               | 判断是否任一元素或所有元素满足条件，只会消耗尽可能少的值。                                      |
| find、rfind            | 返回第一个（或最后一个）满足条件的元素。                                               |
| find_map              | 对找到的元素应用函数并返回 <code>Some</code> 值。                                 |
| position、rposition    | 返回满足条件元素的索引， <code>r</code> 需要迭代器满足 <code>ExactSizeIterator</code> |
| nth、nth_back          | 分别返回从前面或从后面数的第 N 个元素（从 0 开始计数）。                                    |
| last                  | 返回迭代器的最后一个元素，消耗所有元素。                                               |
| partition             | 根据谓词拆分为两个集合：满足条件和不满足条件。                                            |
| extend                | 将迭代器的所有元素追加到已有集合中，需要实现 <code>Extend</code> 特型。                     |

表 4.2: Rust 中常见迭代器方法

## 5 并发

### 5.1 并发

Rust 中的线程安全体系依赖两个核心内置特型: `std::marker::Send` 和 `std::marker::Sync`。

- `Send`: 表示类型的值可以安全地移动到另一个线程。
- `Sync`: 表示类型的不可变引用可以安全地在多个线程之间共享。

fork and join

```
use std::{io, thread};

fn process_files_in_parallel(filenames: Vec<String>) -> io::Result<()> {
    const NTHREADS: usize = 8;
    let worklists = split_vec_into_chunks(filenames, NTHREADS);

    // fork
    let mut thread_handles = vec![];
    for worklist in worklists {
        thread_handles.push(
            thread::spawn(move || process_files(worklist))
        );
    }
    // join
    for handle in thread_handles {
        // std::thread::Result<std::io::Result<()>>
        handle.join().unwrap()?;
    }
    Ok(())
}
```

如果需要在多个线程之间共享同一份数据, 应使用 `Arc<T>` 来实现安全的多线程共享, 每次在创建线程时, 使用 `data.clone()` 传递给此线程。

## rayon

```
use rayon::prelude::*;

// "Do two things in parallel"
let (v1, v2) = rayon::join(fn1, fn2);

// "Do N things in parallel"
giant_vector.par_iter().for_each(|value| {
    do_thing_with_value(value);
});
```

## channel

```
fn start_file_reader_thread(documents: Vec<PathBuf>)
    -> (mpsc::Receiver<String>, thread::JoinHandle<io::Result<()>>)
{
    let (sender, receiver) = mpsc::channel();

    let handle = thread::spawn(move || {
        ...
    });
    (receiver, handle)
}
```

当发送端产生值的速度快于接收端接收和处理值的速度时，通道中未处理的消息会不断积压，为了解决这种背压问题，可以使用同步通道（`std::sync::mpsc::sync_channel`）。

## pipeline

```
fn run_pipeline(documents: Vec<PathBuf>, output_dir: PathBuf)
-> io::Result<()> {
    // Start all 5 stages of the processing pipeline
    let (texts, h1) = start_file_reader_thread(documents);
    let (pints, h2) = start_file_indexing_thread(texts);
    let (gallons, h3) = start_in_memory_merge_thread(pints);
    let (files, h4) = start_index_writer_thread(gallons, &output_dir);
    let result = merge_index_files(files, &output_dir);

    // Wait for all threads to finish
    let r1 = h1.join().unwrap();
    h2.join().unwrap();
    h3.join().unwrap();
    let r4 = h4.join().unwrap();

    // Return the first error encountered (if any)
    // Note: h2 and h3 won't fail
    r1?;
    r4?;
    result
}
```